

Test Report — ocr-engine

Document owner: @quality-assurance Service version: 0.1.0 Last run: 14 September 2026 Result: 39 passed, 0 failed, 21.88 seconds

1. Scope

The system under test is `ocr-engine/`: a single service that accepts a file and returns text together with coordinates and a per-word confidence score. Mapping that text into document fields is not this service's responsibility and is not tested here.

In scope:

Area	How it is tested
Processing path selection	PDF text layer, rasterized PDF, image
File validation	Type by magic bytes, size, page count
Preprocessing	Deskew, denoise, perspective correction, upscale, binarization, morphology
Response shape	Structure, <code>bbox</code> normalization, <code>confidence</code> values, <code>engine</code> per path
Recognized content	Keywords that must appear in the fixture documents
Error handling	Corrupt files, unsupported types, limits exceeded

Out of scope:

Area	Reason
Accuracy on real documents	No golden dataset yet. All fixtures are synthetic
Observability	Structured logging, tracing, and metrics are not implemented
Load and concurrency	No throughput target has been defined
Ingress security	This service has no ingress; strict input validation lives in the edge service

2. Strategy

Three principles shape how the tests are organized:

1. **Layered.** Preprocessing functions are unit-tested with synthetic images whose properties are known in advance (a white square on a black background, text deliberately rotated by 10°). End-to-end paths are tested with fixture files that resemble real documents. Unit tests explain **why** something failed; end-to-end tests show **that** it failed.
2. **Runnable without Tesseract.** Tests that need the Tesseract binary carry the `ocr` marker and are skipped automatically when the binary is missing (`tests/conftest.py`). Contributors without Tesseract can still run the suite, and a Linux CI job remains useful.
3. **Committed fixtures, not regenerated at test time.** Fixtures are produced by `tests/fixtures/generate_fixtures.py` , which needs the Times New Roman font from Windows.

The generated files are committed as the shared baseline; the script documents their origin and makes new variants easy to create.

3. Test Environment

Component	Version
Python	3.9.13
Tesseract	5.4.0.20240606 (leptonica 1.84.1)
OpenCV	4.10.0 (opencv-python-headless)
PyMuPDF	1.26.5
FastAPI	0.128.8
Pydantic	2.13.5
pytest	8.4.2
Operating system	Windows 11

Preprocessing configuration during the run: skew **on**, denoise **on**, perspective correction **on**, upscale to 300 DPI **on**, adaptive binarization **off**, morphology **off**.

4. How to Run

```
pip install -r requirements-dev.txt

pytest                # full suite
pytest -v             # with individual test names
pytest -m "not slow"  # skip full-page OCR (faster)
pytest -m ocr         # only tests that need Tesseract
pytest tests/test_preprocess.py::TestPreprocessSteps::test_deskew_corrects_known_skew
```

Markers in use: `ocr` (requires the Tesseract binary) and `slow` (full-page OCR, takes seconds).

5. Test Cases

5.1 `test_health.py` — 1 case

Case	What it guards
<code>test_health_ok</code>	<code>/health</code> returns 200 with exactly four keys. This catches information leakage if someone later adds internal details to the health response

5.2 test_mime_detection.py — 4 cases

Case	What it guards
test_accepts_allowed_types	JPG, PNG, and PDF are accepted
test_rejects_unknown_signature	A file with unknown magic bytes is rejected. The type is determined from file content, not from the extension or the client-supplied Content-Type
test_rejects_oversized_image	An image > 10 MB is rejected with FILE_TOO_LARGE
test_rejects_oversized_pdf	A PDF > 25 MB is rejected with FILE_TOO_LARGE

5.3 test_pdf_text.py — 4 cases

Case	What it guards
test_pdf_text_layer_creator	A PDF with a text layer yields ocr_source = text_layer , engine = null , and the exact word count. A null engine proves no OCR was run
test_exact_text_match	The letter number and destination are read exactly. This is the advantage of this path: no character is guessed
test_bounding_box_always_normalized	Every bbox lies within 0–1 and confidence is 1.0
test_deny_pdf_without_text	A blank PDF is rejected as having an unusable text layer, so it is routed to OCR instead of returning an empty page

5.4 test_pdf_raster.py — 9 cases

Case	What it guards
test_converts_rgb_to_bgr	Conversion from a PyMuPDF pixmap (RGB) to OpenCV channel order (BGR). Swapped channels do not crash anything; they silently lower accuracy
test_rejects_invalid_pdf	A plain text file is rejected with INVALID_PDF
test_rejects_over_page_limit	Exceeding PDF_MAX_PAGES is rejected, not silently truncated
test_returns_ocr_response	ocr_source = ocr , engine is set, duration_ms > 0
test_reads_document_content	The title, destination, two employee names, and an 18-digit employee ID are recognized
test_reads_document_number	The letter number 090/ST/IX/2026 is read in full, including its slashes
test_rasters_at_configured_dpi	A4 at 300 DPI produces roughly 2480 x 3508 pixels
test_bbox_is_normalized	Every coordinate lies within 0–1
test_reads_every_page	A two-page PDF yields two pages in order, and the second page (an attachment with amounts) is recognized

5.5 test_image_ocr.py — 5 cases

Case	What it guards
<code>test_decodes_png</code>	A PNG decodes into a three-channel array
<code>test_rejects_undecodable_bytes</code>	A file with PNG magic bytes but corrupt content is rejected with <code>UNSUPPORTED_MEDIA_TYPE</code> instead of raising a raw exception
<code>test_returns_ocr_response</code>	Response shape and engine identity
<code>test_recognizes_rendered_text</code>	Text drawn onto an image is recognized back
<code>test_bbox_is_normalized</code>	Every coordinate lies within 0–1

5.6 test_preprocess.py — 16 cases

Unit functions:

Case	What it guards
<code>test_deskew_returns_image_and_angle</code>	Shape of the deskew output
<code>test_deskew_corrects_known_skew</code>	A +10° rotation is detected within 1°
<code>test_deskew_corrects_negative_skew</code>	A −7° rotation is detected. Guards the handling of angle sign, a classic source of bugs with <code>minAreaRect</code>
<code>test_deskew_leaves_horizontal_text_untouched</code>	A straight image is returned identical , with no 0° rotation that would still blur pixels through interpolation
<code>test_denoise_returns_same_shape</code>	Denoising does not change dimensions or data type
<code>test_adaptive_threshold_returns_binary</code>	Output contains only 0 and 255
<code>test_upscale_to_dpi_increases_size</code>	Upscaling to 300 DPI enlarges the image
<code>test_upscale_to_dpi_no_change_when_scale_le_1</code>	A target lower than the source does not shrink the image
<code>test_morphological_cleanup_returns_same_shape</code>	Morphology does not change dimensions
<code>test_perspective_restores_aspect_ratio</code>	A page photographed at an angle is restored to its original aspect ratio within 15%

Orchestration:

Case	What it guards
<code>test_returns_preprocess_result</code>	All result attributes are available for diagnostics
<code>test_steps_applied_contains_enabled_steps</code>	Enabled steps are recorded in <code>steps_applied</code>
<code>test_input_output_resolution_recorded</code>	Resolution before and after is recorded
<code>test_duration_ms_positive</code>	Duration is measured
<code>test_output_image_is_binary_after_threshold</code>	With binarization on, the output is truly binary
<code>test_threshold_and_morphology_skipped_when_disabled</code>	Configuration switches really turn their steps off

6. Manual Testing on Crumpled Documents

Outside the automated suite, two photo fixtures of a crumpled document were sent to the `POST /ocr` endpoint to observe behavior under poor conditions. The lightly crumpled result served as the baseline and was compared with the heavily crumpled one.

Section	Light crumple	Heavy crumple	Impact	Severity
Letterhead phone number	3449230,	3449230:	Cosmetic	Low
Letterhead rule	Not recognized	Recognized as a run of symbols	Garbage token	Low
Employee 1 grade	III-b	1II-D	Invalid value	High
Employee 2 grade	III-c	III-€	Invalid value	High
Date labels	Tanggal , Kembali	Tanggal , Kembati	Parser anchor lost	Medium
Trip duration	: 4 (empat) hari	"4 Cempat) hari	Separator lost	Medium
Budget source	DIPA Ditjen ...	Word DIPA missing	Wrong value with no signal	Critical

Recognized correctly in both photos: every employee ID, every date, every name, and the letter number.

Three findings that affect system design

1. **confidence cannot be used to flag errors.** The misread `3449230:` scored 0.90, while the correct `Pangkat/GoLongan` scored 0.48 and the correct `Santoso` scored 0.53. The value measures how confidently glyph shapes were recognized, not whether the content is right. A "needs review" flag in the application must not rely on this number alone.
2. **A missing word is the most dangerous failure.** `Ditjen Perbendaharaan TA 2026` looks plausible, so nothing downstream can tell that `DIPA` was ever there. The fix belongs in preprocessing, not in text post-processing.
3. **Adaptive binarization is harmful in the current configuration.** With a fixed `blockSize` of 11 on an image already upscaled roughly four times, the interior of bold strokes turns white. This is why `PREPROCESS_THRESHOLD` is off by default and Tesseract is left to binarize on its own.

7. Limitations of This Report

Fixtures do not measure accuracy. Every test file was generated programmatically with one layout and one font, with no stamps, no logo letterhead, no yellowed paper, and no handwriting. Fixtures cover code paths and lock in regressions — a fully passing suite does **not** mean the system is accurate enough.

A trustworthy accuracy figure can only come from real documents. Collecting real sample documents is still an open question in the product requirements, and it is a prerequisite before any accuracy claim can be made.

All fixture data is fictional: names, employee IDs, and letter numbers were invented for testing.

8. Recommendations

#	Recommendation	Rationale
1	Collect a golden dataset of real documents with reference transcriptions	Without it, no accuracy figure can be defended
2	Measure accuracy per field, not only character error rate	One wrong character in an employee ID costs far more than one in a free-text description
3	Compare preprocessing variants side by side: true source resolution, no binarization, illumination normalization	These three levers matter most for crumpled and shadowed photos
4	Add tests for observability once it is implemented	Including a check that OCR text never reaches the logs
5	Add a page-limit test for the text layer path	The limit is currently only enforced on the raster path
6	Run load tests once a throughput target is defined	OCR is CPU-bound; behavior under concurrent requests is unknown